

MODULE 8

Anomaly Detection

Detecting the faults you never recorded

TinyML Course · Day 3

Classifiers always answer — even when they shouldn't

- A softmax classifier with 5 classes **always** outputs 5 probabilities summing to **1.0**
- Feed it something it has never seen → it still picks a class, often confidently

input: bearing wear (never recorded)

output: normal 0.09 | imbalance 0.71 | blocked 0.12 | scrape 0.05 | off 0.03

↑ confident. wrong.

ML models *always* predict *something*, no matter what data you throw at them.
— old course exercise, still the truth

The open-world problem in condition monitoring

Trained states: `normal · imbalance · blocked · scrape · off`

Reality also contains:

- bearing wear (your "worn fan" — we kept it out of training on purpose)
- loose mounting / resonance
- dust build-up, voltage sag, cable rubbing the blades...

You cannot record every fault in advance. But you can record *normal* for as long as you like — and flag everything that stops looking like it.

Anomaly detection — flip the problem

- **Supervised classifier:** "which of the *known* states is this?"
- **Anomaly detector:** "how far is this from *everything I saw during training*?"

Recipe (all methods today):

1. Train on **normal data only** (cheap, abundant)
2. Compute an **anomaly score** for each new window
3. Pick a **threshold** — above = anomaly

Three ways to build the score: **distance**, **likelihood**, **reconstruction error**.

Method 1 — K-means distance scoring

1. Fit **K cluster centroids** to normal feature vectors (scikit-learn `KMeans`)
2. Score of a new window = **distance to its nearest centroid**
3. Far from every centroid → anomaly

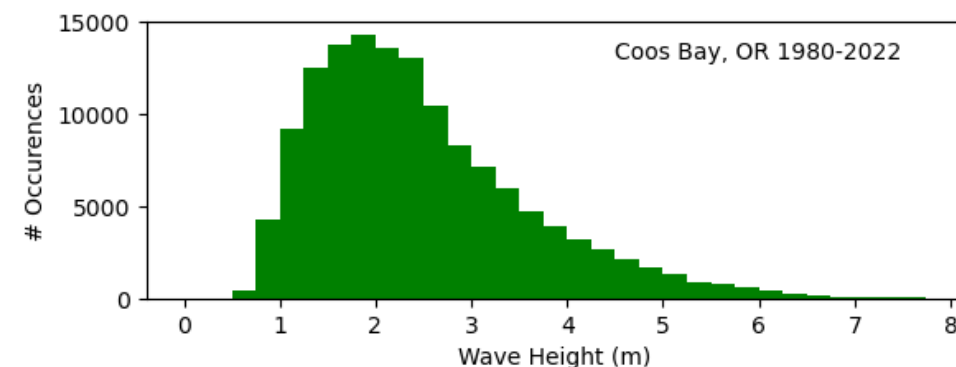
```
kmeans = KMeans(n_clusters=3).fit(X_normal)
d = np.min(np.linalg.norm(X_new - kmeans.cluster_centers_, axis=1))
```

- On-device cost: $K \times n_{\text{features}}$ multiply-adds — **microseconds**
- This is exactly Edge Impulse's *K-means Anomaly Detection* block

Picking the threshold

```
distances = [dist_to_nearest_centroid(x) for x in X_normal]
threshold = np.percentile(distances, 98) # start at the 98th percentile
```

- Plot the histogram of *normal* scores — threshold sits at its right tail
- Then tune against reality:
 - threshold ↓ → more sensitivity, more **false alarms**
 - threshold ↑ → fewer alarms, more **missed faults**
- There is no correct value — only a cost trade-off you must own



Where K-means breaks

- Distance-to-centroid draws **spherical** boundaries
- Real feature clouds are often **elongated and correlated** (e.g. RMS_x grows with RMS_y)
- A point can be *inside* the true normal cloud but *far* from a centroid — false alarm
- And vice versa: an anomaly can hide *between* two centroids

Notebook demo: K-means on elliptic blobs draws visibly wrong borders.

"K-means is not really good at this — now what...?" → give the clusters a shape.

Method 2 — Gaussian Mixture Models

- Model normal data as a **mixture of K Gaussians** — each with mean *and covariance* → ellipses, not spheres
- Score = **log-likelihood**: `gm.score_samples(X_new)`
- Low likelihood = low-density region = anomaly

```
gm = GaussianMixture(n_components=3, n_init=10).fit(X_normal)
threshold = np.percentile(gm.score_samples(X_normal), 2)
is_anomaly = gm.score_samples(X_new) < threshold
```

`covariance_type` : `spherical` ($\approx$ K-means) → `diag` → `tied` → `full` — flexibility vs parameters.

GMM fits on the MCU too

- Inference = evaluate K Gaussians: a handful of matrix–vector products
- **emlearn supports GMM** (`eml_mixture`) — train in scikit-learn, convert to C, same workflow as the Random Forest
- Edge Impulse also has a **GMM anomaly block** next to the K-means one

Rule of thumb: start with K-means; reach for GMM when normal data is clearly multi-modal or correlated.

Method 3 — Autoencoder reconstruction error

- Train a network to **reproduce its own input** through a constrained middle layer
- It can only do that well for data that looks like the training data
- Score = **reconstruction error** $\text{MSE}(x, \hat{x})$

raw window (200×1) → Conv1D encoder → latent 25×64 feature map → Conv1D decoder → $\hat{x}$
score = $\text{mean}((x - \hat{x})^2)$

- The squeeze here is **temporal**: 200 time steps → 25 (a dense AE would instead use a genuinely narrow layer, e.g. 200 → 16 → 200)
- Trained **only on normal windows** — faults were never compressible

What the autoencoder sees

- Normal test windows: $MSE \approx 0.002$ — reconstruction hugs the signal
- Anomalous windows (shifted frequency, spikes): MSE 10–100× higher
- The decoder redraws every input as *the nearest normal-looking signal* — the residual is the fault signature

Threshold recipe is identical: percentile of **normal** reconstruction errors.

(Notebook: synthetic 50 Hz "vibration" vs 80 Hz + spikes.)

Three tools, one job

	K-means	GMM	Autoencoder
Score	distance	likelihood	reconstruction error
Input	features	features	raw windows (or features)
Captures	blobs	elongated/multi-modal	arbitrary structure
Train cost	seconds	seconds	minutes
MCU cost	~ μ s, bytes	~ μ s, KB	ms, tens–hundreds of KB
Today's role	deploy on device	notebook + emlearn option	notebook (concept)

The notebook AE (65 k params $\approx$ 254 KB float32) claims **a quarter of our 1 MB flash** for weights alone — plus tens of kB of RAM for Conv1D activations. Size before you ship.



Lab track 1 — the three methods in notebooks (~40 min)

`exercises/module8-anomaly/notebooks/` — in order:

- `01_kmeans_anomaly.ipynb` → `02_gmm_anomaly.ipynb` → `03_autoencoder_anomaly.ipynb`
- All three use the same protocol: train on `normal` **only**, score everything
- Hold out one fault class entirely — does each method still catch it?
- **Done when:** you have three ROC-ish score plots and a favourite method

Edge Impulse anomaly blocks

The screenshot displays the Edge Impulse IDE interface for 'Impulse #1'. At the top, a purple header bar contains the title 'Impulse #1' and a descriptive text box: 'An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data.' Below this, the workspace is divided into several colored blocks:

- Time series data (Red block):** Contains settings for 'Input axes (3)' (x, y, z), 'Window size' (2,000 ms), 'Window increase (stride)' (25 ms), 'Frequency (Hz)' (50), 'Zero-pad data' (checked), and 'Train on data subset' (100 %).
- Spectral Analysis (White block):** Has a 'Name' field set to 'Spectral features' and 'Input axes (3)' (x, y, z) all checked.
- Classification (Purple block):** Has a 'Name' field set to 'Classifier' and 'Input features' set to 'Spectral features' (checked). Its 'Output features' are '4 (Idle, Lift, Maritime, Terrestrial)'.
- Anomaly Detection (K-means) (Purple block):** Has a 'Name' field set to 'Anomaly detection' and 'Input features' set to 'Spectral features' (checked). Its 'Output features' are '1 (Anomaly score)'.
- Output features (Green block):** Displays the final output: '5 (Idle, Lift, Maritime, Terrestrial, Anomaly score)'.

A 'Save Impulse' button is located at the bottom right of the workspace. A dashed box at the bottom center contains a lightning bolt icon and the text 'Add a processing block'.

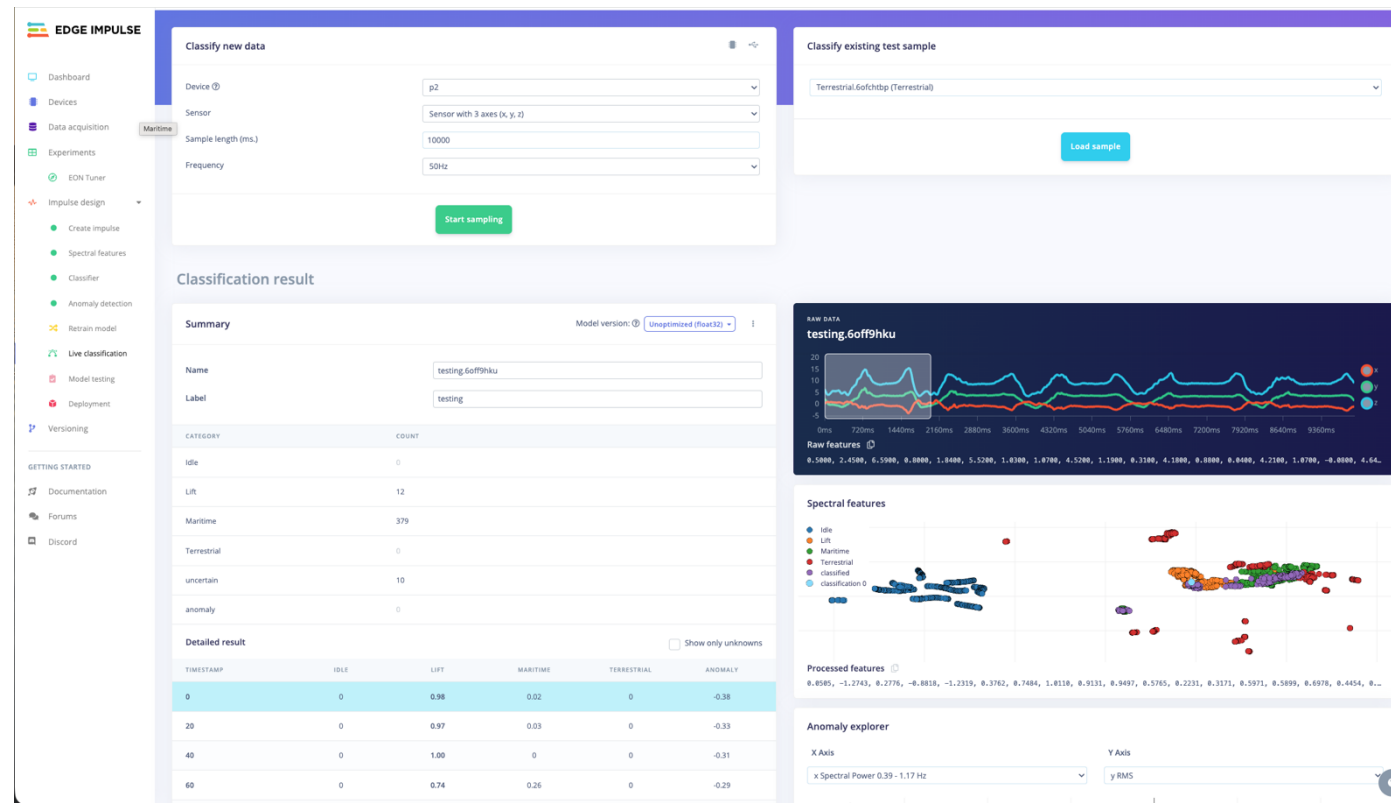
- Add **Anomaly Detection (K-means)** as a *second* learning block in your Module 7 impulse
- Classifier and anomaly detector share the same DSP features

Configuring the block

- Select **which features** the K-means runs on — start from the feature-importance list (Module 5 intuition!)
- For the fan: RMS + spectral-power axes usually separate best
- Cluster count: default 32 is fine to start; fewer clusters = tighter "normal" definition
- **Train on** `normal` **(and** `off` **) data only** — remove fault classes from the anomaly training set

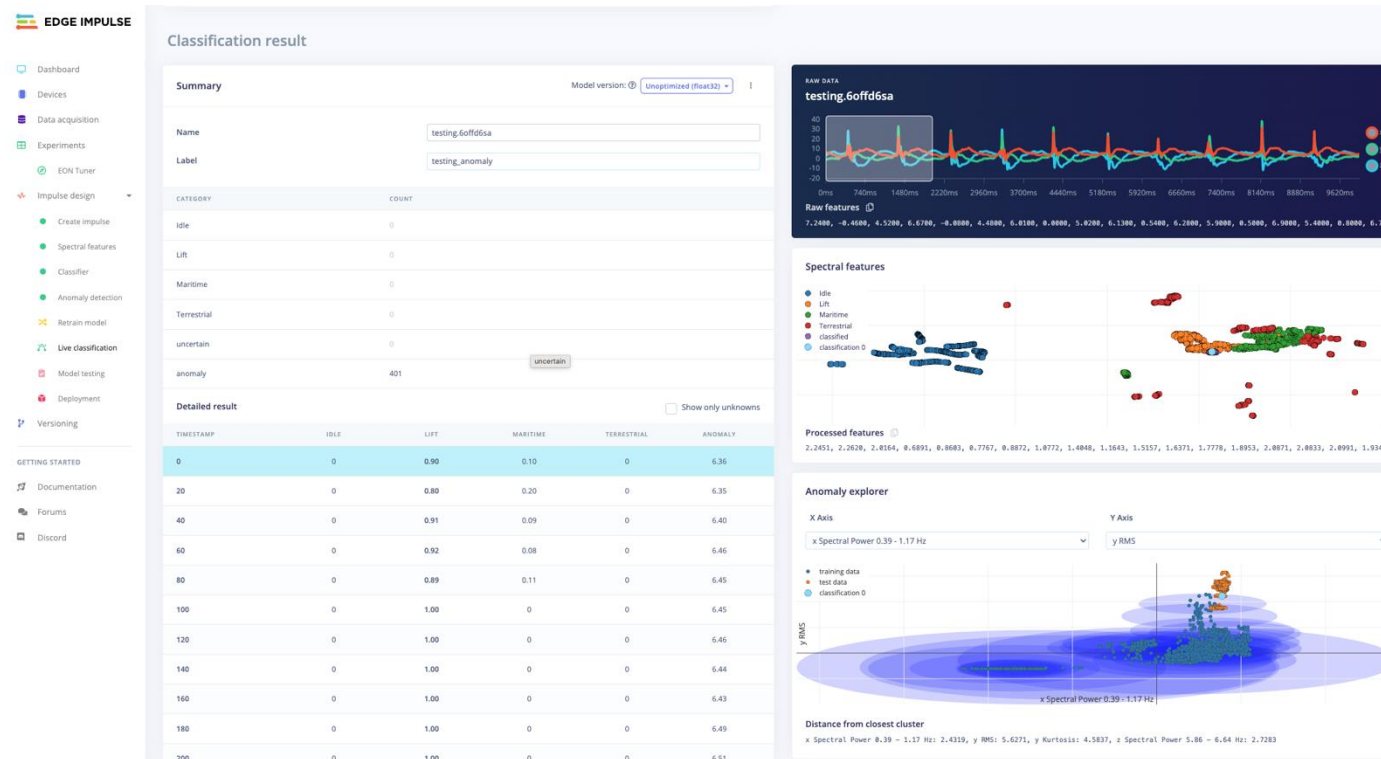
Verify in the Studio first

Known state → classification confident, **anomaly score low**:



...then show it something new

Unknown state → **anomaly score jumps:**



Lab version: hold back one fault state (e.g. *scrape* or the worn fan) from **all** training, then replay it in *Live classification*.

On-device pattern: anomaly score as a gate

```
run_classifier(&signal, &result, false);

if (result.anomaly > ANOMALY_THRESHOLD) {    // tune from Studio live tests
    Serial.printf("ANOMALY score=%.2f (classifier output suppressed)\n",
        result.anomaly);
    digitalWrite(LED_BLUE, HIGH);           // unknown-state indicator
} else {
    print_best_class(&result);               // trust the classifier
    digitalWrite(LED_BLUE, LOW);
}
```

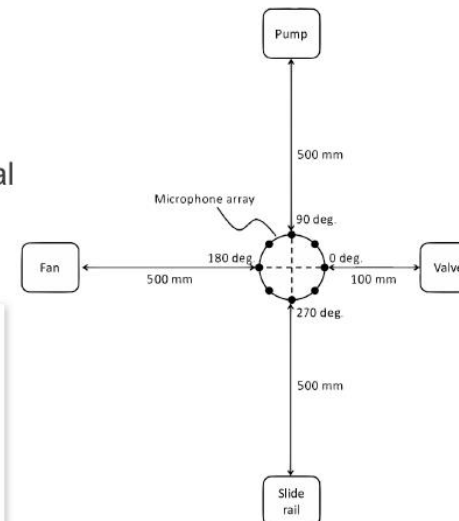
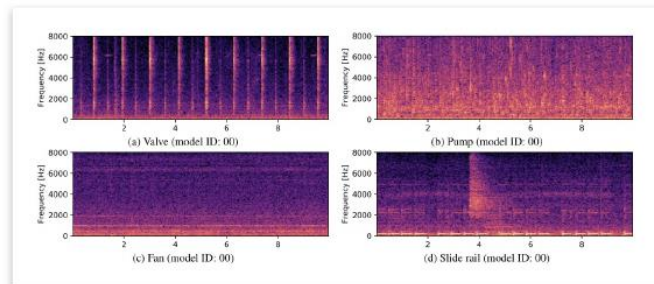
- `result.anomaly` is populated automatically when the impulse has an anomaly block
- Threshold: start where Studio's live tests separated known/unknown, then tune on the rig

This is a live research field — MIMII

- **MIMII**: Sound Dataset for **Malfunctioning Industrial Machine** Investigation
- Real factory recordings: **fans**, pumps, valves, slide rails — normal + faulty
- The published baseline approach: train an **autoencoder on normal sound only** — exactly today's method 3

Example: MIMII Dataset

- **Sound** Dataset for Malfunctioning Industrial Machine Investigation (MIMII)
- **Anomaly Detection**



Purohit, Harsh, et al. "MIMII dataset: Sound dataset for malfunctioning industrial machine investigation and inspection." (2019).

From fan rig to plant floor

- Production condition monitoring is **anomaly-first**: every installation gets its own "normal" baseline
- Classifiers come second — they explain *known* faults; anomaly scores catch the rest
- Re-baseline after maintenance (new bearing = new normal)
- Trend the score, don't just threshold it: slow drift = wear, step change = event



Lab track 2 — anomaly detection on the device

[exercises/module8-anomaly/](#) → Track 2

(track 1 — the notebooks — ran after the three-methods section)

- Add the K-means anomaly block to your Module 7 EI project
- Retrain on **normal-only**, redeploy the Arduino library
- Print `result.anomaly`, gate the classifier on it

Done when: the held-out fault lights the anomaly LED — while trained states still classify.

Write your threshold + observed scores into the README table.

Bridge to Module 9

Your device now runs: **DSP features** → **classifier** → **anomaly gate** — all in float32.

Questions we have not asked yet:

- Does it *have* to be float32? (No.)
- What does int8 cost in accuracy — and buy in RAM/flash/latency?
- Are all those weights even *doing* anything? (Also no.)

Next: Module 9 — quantisation & pruning. Same models, smaller & faster.